TRƯỜNG ĐẠI HỌC PHENIKAA

Khoa Điện-Điện tử

BÁO CÁO BÀI KIỂM TRA SỐ 2

Thiết kế số nhỏ và quy trình từ register-transfer level (RTL) đến Graphic Database System II (GDSII) bằng công cụ nguồn mở

Giảng viên hướng dẫn: Đinh Văn Nam

Học phần	EEE703128 — Nhập môn Thiết kế Vi mạch Tích hợp
Mã học phần	EEE703128 · 3 tín chỉ 3(3;0;6)
Tên bài	Bài kiểm tra số 2 — Báo cáo cá nhân RTL-to-GDSII
Mã đề · Tên đề tài	I05 – Bộ tính sin và cos 8 bit bằng coordinate rotation digital computer (CORDIC)
Họ và tên sinh viên	Trương Hải Minh
Mã số sinh viên	24100706
Lớp	Nhập môn thiết kế vi mạch tích hợp-1-1-26(N02)
Giảng viên hướng dẫn	Đinh Văn Nam
Kho GitHub	`https://github.com/haiminhtruongisavailable/I05_Intro_to_chip_design_project`
Mã commit nộp bài	0520dd2
Lần chạy continuous integration (CI) dùng cho báo cáo	None
Ngày nộp	28/10/2026

Mục lục

1 Đặt vấn đề và đặc tả **4**

1.1 Phát biểu bài toán 4

1.2 Giả định và trường hợp biên 5

1.3 Luật tổ hợp và sơ đồ trạng thái 6

2 Thiết kế RTL **8**

2.1 Vị trí của mục 8

2.2 Sơ đồ khối 8

2.3 Quyết định thiết kế 8

2.4 Đoạn cốt lõi 9

3 Kiểm chứng chức năng **11**

3.1 Vì sao chọn từng ca 11

3.2 Giá trị mong đợi 11

3.3 Hai ca tự chọn 11

3.4 Dạng sóng 11

3.5 Ca từng đỏ 12

4 Kết quả tổng hợp **14**

4.1 Ước lượng trước khi đối chiếu 14

4.2 Đối chiếu flip-flop 14

4.3 Loại cell 14

4.4 Những gì thiết kế đặt vào chuỗi nhịp 16

5 Kết quả phân tích thời gian **17**

5.1 Setup và hold 17

5.2 Đường tới hạn 17

5.3 Tần số ước tính 17

5.4 Mạch có thanh ghi 18

5.5 Không hạ chu kỳ 18

6 Công suất và kiểm tra vật lý **19**

6.1 Công suất 19
6.2 Luật vẽ và đối chiếu sơ đồ 19
6.3 Luật vẽ không chứng minh chức năng 19

7 Quan sát layout **20**
7.1 Ảnh layout 20
7.2 Những gì nhìn thấy 20
7.3 Liên hệ quy trình 21
7.4 Phương pháp của lần chạy 21

8 Phần đóng góp của AI **24**

9 Kết luận, giới hạn và tự đánh giá **25**
9.1 Điều kiện của kết luận 25
9.2 Việc chưa có số 25
9.3 Điều không được kết luận 25
9.4 Chip không được chế tạo 25

A Mã nguồn RTL đầy đủ **27**

B Testbench đầy đủ **44**

C Thông tin tái lập **51**

1 Đặt vấn đề và đặc tả

1.1 Phát biểu bài toán

Báo cáo này đi từ register-transfer level (RTL) đến Graphic Database System II (GDSII) của một mạch tính sine và cosine. Mạch nhận một mã góc vào và trả `sin` cùng `cos`, mỗi cái một byte có dấu. Mã góc là một byte không dấu, và một mã bằng 360°/256, nên 256 mã phủ một vòng. Xung `start` nhận đúng mã đang có trên chân vào, và mã đó là góc lấy mẫu ban đầu, chốt một lần ở nhịp `LOAD`. Khi lần tính kết thúc, mạch giữ hai byte có dấu: mã 127 đứng gần +1, còn mã −128 đứng gần −1.

Phép tính là coordinate rotation digital computer (CORDIC) 8 bit ở chế độ quay. Mạch không nhân và không đọc bảng sine–cosine. Trước tám bước `RUN`, mạch gấp góc lấy mẫu ban đầu về một phần tư và lưu `sign_sin` cùng `sign_cos`. Góc trong quá trình xử lý là `z`, tức phần còn lại của mã đã chốt, không phải một lần chốt góc mới. Mỗi nhịp `RUN` chỉ cộng hoặc trừ một `alpha` vào `z`. `sign_sin` và `sign_cos` chỉ tác động khi ghi `sin` và `cos`.

Mạch dừng sau mỗi lần tính và không tự cộng dồn mã góc. Sóng của lần tính đó gồm nhiều cạnh `clk`: một nhịp `LOAD`, tám nhịp `RUN` và một nhịp `SAMPLE`. Đó không phải nhiều xung `start`. `rst_n` đưa trạng thái về `IDLE`. Bảng 1.1 kê tám tín hiệu của lần trao đổi này.

Bảng 1.1 — Bảng tín hiệu vào/ra

Tín hiệu	Chân	Độ rộng	Chiều	Mức tích cực	Ý nghĩa
nhịp	`clk`	1	vào	cạnh lên	đẩy một bước trạng thái
reset	`rst_n`	1	vào	thấp	reset bất đồng bộ, đưa về `IDLE`
góc	`ui_in[7:0]`	8	vào	mức	mã góc vào, chốt ở `LOAD`
khởi động	`uio_in[0]`	1	vào	cao	chỉ nhận khi đang `IDLE`
sine	`uo_out[7:0]`	8	ra	có dấu	sine, giữ đến lần `start` kế
cosine	`uio_out [7:0]`	8	ra	có dấu	cosine khi có kết quả
cho phép ra	`uio_oe[7:0]`	8	ra	cao	bật cả byte khi trả cosine
xong	nội bộ	1	nội bộ	cao	một nhịp, không đưa ra chân vỏ

1.2 Giả định và trường hợp biên

Bảng 1.2 ghi tám giả định đã chốt trước khi lấy số chạy. `z` là góc trong quá trình xử lý, cùng đơn vị với mã góc vào. `z` được ghi một lần từ mã đã gấp, rồi chỉ đổi theo `alpha`. Mọi mã từ 0 đến 255 đều hợp lệ, nên không có mã vào bị loại. Mã 255 đứng một bước trước đủ một vòng. 30° và 60° không rơi đúng một mã. Mạch dùng mã gần nhất, tức mã 21 và mã 43.

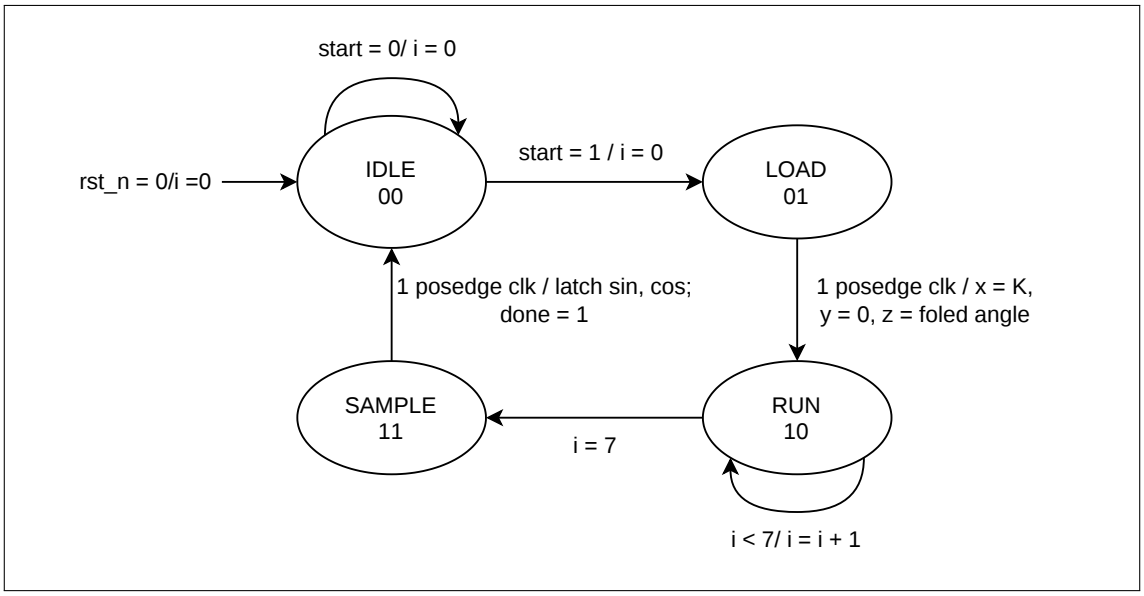
So sánh `z` với 0 lấy trường hợp bằng 0 là không âm. $z \geq 0$ quay một phía và $z < 0$ quay phía kia, nên mã góc 0 vẫn có bước quay đầu. Sau phép dịch, `sin` và `cos` bị kẹp trong −128 đến 127. Đảo dấu −128 không ra +128, vì byte có dấu không chứa số đó. Ngay sau `rst_n`, trạng thái là `IDLE` và hai byte kết quả bằng 0. `start` chỉ được đọc ở `IDLE`. Ở `LOAD`, `RUN` và `SAMPLE`, `start` không đổi trạng thái.

Bảng 1.2 — Giả định và trường hợp biên

#	Giả định / trường hợp biên	Xử lý theo đặc tả
1	mọi mã 0–255	đều là góc hợp lệ, không loại mã vào
2	`z` = 0, kể cả mã góc 0	quay như `z` không âm, vẫn có bước đầu
3	30°, 60°	lấy mã gần nhất, 21 và 43
4	mã 255	một bước trước 360°, góc biên của dải vào
5	kết quả ngoài −128..127	kẹp vào hai đầu thang
6	đảo dấu −128	byte có dấu không chứa +128
7	ngay sau `rst_n`	`IDLE`, `sin` và `cos` bằng 0
8	`start` ngoài `IDLE`	không đổi trạng thái. Nhịp `done` đã là `IDLE`, nên `start` ở đó mở lần sau

1.3 Luật tổ hợp và sơ đồ trạng thái

Máy có bốn trạng thái, mã hai bit. `IDLE` là 00, `LOAD` là 01, `RUN` là 10 và `SAMPLE` là 11. Ba đầu vào là `clk`, `rst_n` và `start`. Hai biến trạng thái là `state`, rộng 2 bit, và `i`, rộng 3 bit. Bốn đầu ra là `load`, `run`, `sample` và `done`. `load`, `run` và `sample` lần lượt bằng 1 khi `state` là `LOAD`, `RUN` và `SAMPLE`. `done` là `sample` ghi trễ một cạnh `clk`. Hình 1 là sơ đồ trạng thái của máy này.



Hình 1: Sơ đồ trạng thái: `IDLE` 00, `LOAD` 01, `RUN` 10, `SAMPLE` 11, với `rst_n`, `start` và `i` trên các cạnh.

`rst_n` thấp đưa `state` về 00, `i` về 0 và `done` về 0. Ở `IDLE`, `start` bằng 0 thì máy ở lại và `i` giữ nguyên. `start` bằng 1 thì nhịp sau là `LOAD` và `i` về 0. `LOAD` kéo dài một `clk` và luôn sang `RUN`.

Ở `RUN`, `i` nhỏ hơn 7 thì máy ở lại và `i` tăng 1. `i` bằng 7 thì nhịp sau là `SAMPLE` và `i` giữ nguyên. `SAMPLE` kéo dài một `clk` và luôn về `IDLE`. `done` bằng 1 ở `IDLE` ngay sau `SAMPLE`.

`RUN` chiếm tám cạnh `clk`, nên một vòng sau `start` là một `LOAD`, tám `RUN` và một `SAMPLE`. Cạnh ra khỏi `LOAD` ghi `K` vào `x`, 0 vào `y`, góc đã gấp vào `z`, cùng `sign_sin` và `sign_cos`. Cạnh ra khỏi `SAMPLE` ghi `sin` và `cos`. Bước `i` bằng 7 không đổi `z` vì `alpha` bằng 0.

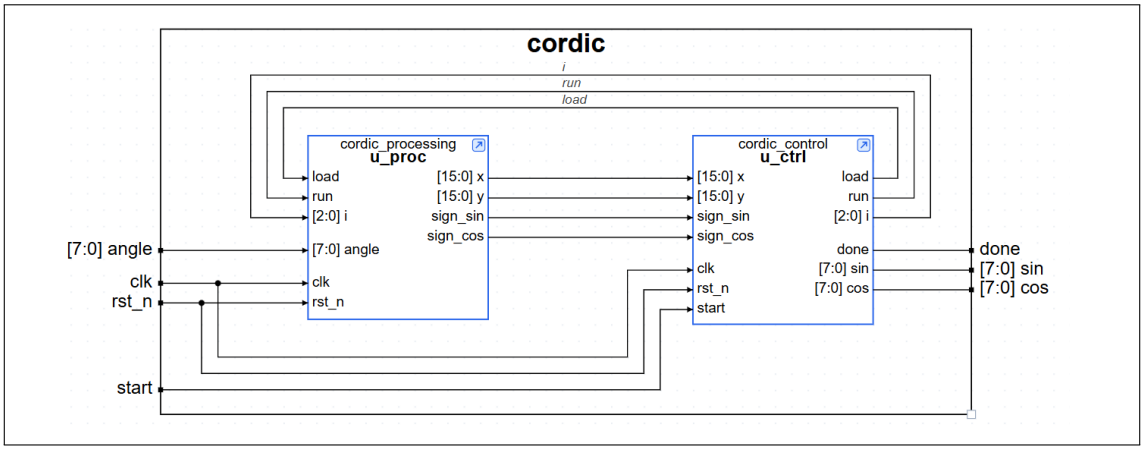
2 Thiết kế RTL

2.1 Vị trí của mục

Lõi chia thành khối điều khiển và khối xử lý. Khối điều khiển giữ bốn trạng thái. Khối xử lý giữ đường `x`, `y` và `z`. Số cell và slack bắt đầu từ Mục 4. Mã đầy đủ nằm ở Phụ lục A.

2.2 Sơ đồ khối

Ba hình sau được chụp từ schematic editor của thiết kế `cordic`. Slide `top` là cả lõi, với khối xử lý cạnh khối điều khiển. Slide `pu` là đường `x`, `y` và `z`. Slide `cu` là bốn trạng thái cùng các thanh ghi kết quả.



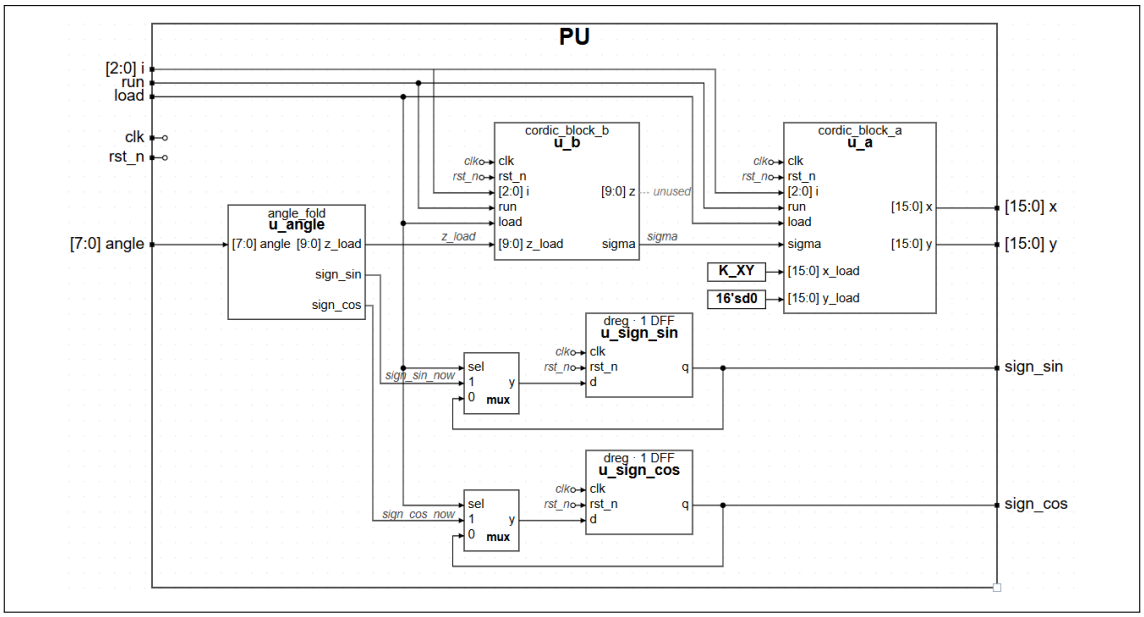
Hình 2: Slide `top`: lõi `cordic`, khối xử lý cạnh khối điều khiển.

2.3 Quyết định thiết kế

Bảng 2.1 ghi bốn quyết định của lõi. Phép quay đi bằng tám bước cộng, trừ và dịch trên một bộ cộng. Một phép nhân rộng làm cùng việc trong một nhịp. Phép đó cần cell nhân, nên thiết kế đổi cell nhân lấy thêm nhịp `clk`. `x` và `y` rộng 16 bit. `K` bằng 9949 không nằm trong 8 bit. Đường 8 bit không chứa được giá trị đầu của phép quay.

Góc vào được gấp về một phần tư trước khi quay. Một đường dữ liệu vì thế phủ cả vòng, và bốn mạch quay song song không có trong sơ đồ. Bốn trạng thái `IDLE`, `LOAD`, `RUN` và `SAMPLE` dùng lại một bộ cộng trong tám nhịp. Trải tám bước thành mạch tổ hợp sẽ đặt nhiều bộ cộng vào cùng một nhịp.

Bảng 2.1 — Các quyết định thiết kế



Hình 3: Slide `pu`: đường `x`, `y` và `z` trong khối xử lý.

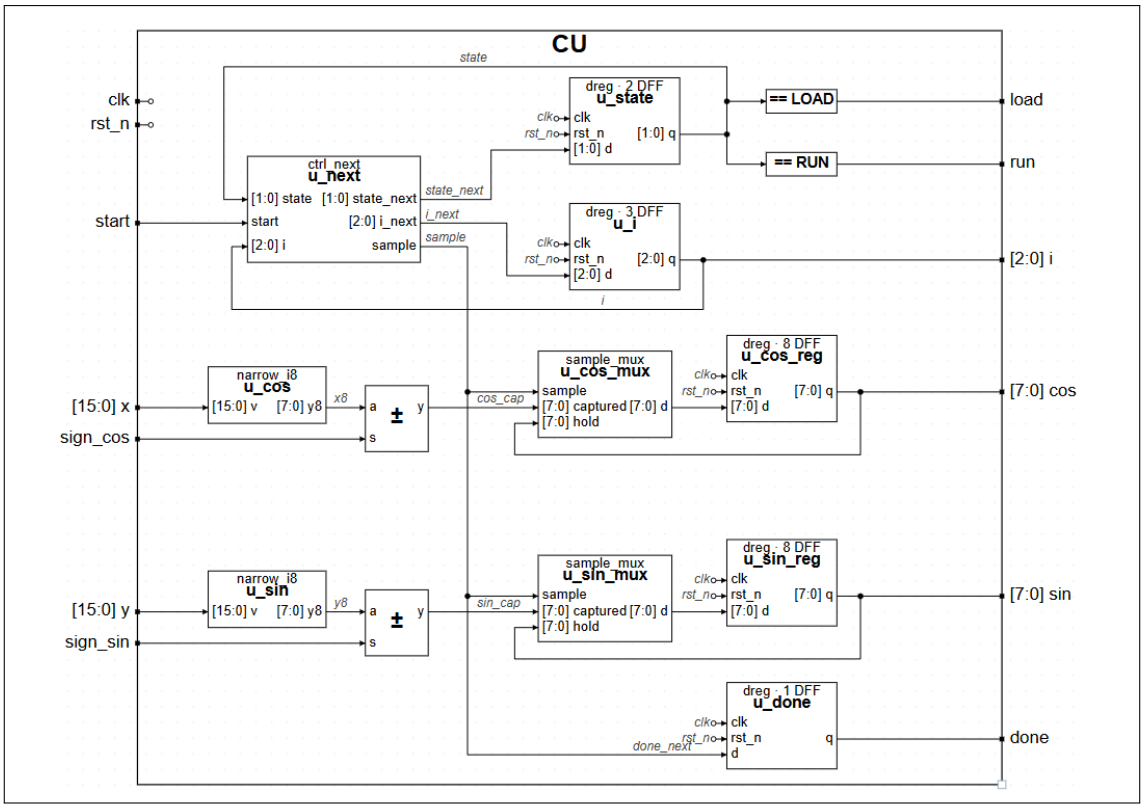
#	Quyết định	Phương án khác đã cân nhắc	Lý do chọn
1	tám bước cộng, trừ, dịch	một phép nhân rộng	bỏ cell nhân, đổi lấy nhiều nhịp
2	`x`, `y` 16 bit	`x`, `y` 8 bit	`K` = 9949 không nằm trong 8 bit
3	gấp góc về một phần tư	bốn mạch quay song song	một đường dữ liệu cho cả vòng
4	bốn trạng thái, dùng lại bộ cộng	trải tám bước thành mạch tổ hợp	một bộ cộng, tám nhịp

2.4 Đoạn cốt lõi

Trên slide `cu`, `u_next` là hàm `ctrl_next` và `u_state` là thanh ghi 2 bit của `state`. `u_i` là thanh ghi 3 bit của `i`. Hàm này đọc `state`, `i` và `start`, rồi ghi `state_next` cùng `i_next`. `eq_load` kéo `load` lên 1 khi `state` là `LOAD`. `eq_run` kéo `run` lên 1 khi `state` là `RUN`.

`axis_mux`, `addsub_block` và `axis_reg` nằm trong `cordic_cells.v`. `sel` của `axis_mux` là {`load`, `run`}. `load` chọn giá trị nạp. `run` chọn giá trị vừa quay. Cả hai bằng 0 thì mux giữ thanh ghi. `addsub_block` cộng khi `sigma` khớp `add_on_sigma`, và trừ khi hai tín hiệu đó khác nhau.

Trên slide `pu`, `u_b` cập nhật `z`. `u_angle` gấp `angle` thành `z_load`. Khi `load` bằng 1, mux của `u_b` lấy `z_load`. Khi `run` bằng 1, `addsub_block` với `add_on_sigma` bằng 0 trừ `alpha` khỏi `z` nếu `sigma` bằng 1, và cộng `alpha` nếu `sigma` bằng 0. `atan_lut` chọn `alpha`



Hình 4: Slide `cu`: bốn trạng thái và các thanh ghi kết quả.

theo `i`. `axis_reg` ghi kết quả vào `z` ở cạnh `clk`.

`u_a` cập nhật `x` và `y` bằng `shift_block`. `sigma` bằng 1 thì `x` trừ `y` $\gg$ `i` và `y` cộng `x` $\gg$ `i`. `add_on_sigma` của đường `x` là 0, và của đường `y` là 1. Khi `load` bằng 1, `x` nhận 9949 từ `const_K_XY` và `y` nhận 0 từ `const_16_sd0`.

Phụ lục A chép nguyên văn các file của lõi, của vỏ chân và của testbench.

3 Kiểm chứng chức năng

3.1 Vì sao chọn từng ca

Bảng 3.1 có mười mã, chia bốn nhóm. Năm mã đầu phủ 0°, 30°, 45°, 60° và 90°. Hai mã tự chọn là 96 và 255. Mã 96 là 135°. Sau khi gấp, `z_load` bằng 32, cùng giá trị của mã 32, và chỉ `sign_cos` bằng 1, nên ca này kiểm tra dấu cosine trên đúng phép quay của 45°. Mã 255 là biên, vì nó đứng một bước trước đủ một vòng. Ba mã 128, 192 và 224 thêm một mã cho mỗi phần tư còn lại.

3.2 Giá trị mong đợi

Kỳ vọng được tính trong `error_table.py` trước khi trừ số của mạch. Với mã n,

$$\theta = 2\pi n/256, \qquad \text{ideal} = \text{round}(128 \sin\theta).$$

Cosine dùng cùng công thức với $\cos\theta$. Mỗi kết quả bị kẹp vào -128 đến 127.

3.3 Hai ca tự chọn

Mã 96 là 135° đúng. `z_load` sau gấp bằng 32, như mã 32. `sign_sin` giữ 0 và `sign_cos` bằng 1. Phép quay trong `u_b` vì thế đi lại đường của 45°, và `SAMPLE` chỉ đổi dấu `cos`. Mã 255 là 358,59°, một bước trước đủ một vòng.

3.4 Dạng sóng

`tb_gtkwave.v` chạy một lần tính với mã góc vào 32. Mã này không phải 32°. Góc lấy mẫu ban đầu là $32 \times 360°/256 = 45°$.

Hình 5 là sóng của lần tính đó. Ảnh gồm `state`, `i`, `x`, `y`, `z`, `sigma`, `sin`, `cos` và `done`. Bảng dưới là phép tính tay của tám bước `RUN`. Mỗi hàng `RUN` ghi giá trị ở đầu bước, và hàng kế là kết quả của bước đó. Khi $\texttt{sigma} = 1$, tức $\texttt{z} \geq 0$, một bước là

$$\texttt{x} \leftarrow \texttt{x} - (\texttt{y} \gg \texttt{i}), \quad \texttt{y} \leftarrow \texttt{y} + (\texttt{x} \gg \texttt{i}), \quad \texttt{z} \leftarrow \texttt{z} - \texttt{alpha}.$$

Khi $\texttt{sigma} = 0$, cả ba phép đổi chiều. Tám giá trị `alpha` là 32, 19, 10, 5, 3, 1, 1 và 0.

Trạng thái	i	sigma	x	y	z	Việc của bước
IDLE, sau rst_n	0	1	0	0	0	sin và cos bằng 0
LOAD	0	1	0	0	0	start còn cao
RUN	0	1	9949	0	32	vừa ghi K và z từ mã góc vào
RUN	1	1	9949	9949	0	xong bước 0
RUN	2	0	4975	14923	−19	xong bước 1
RUN	3	0	8705	13680	−9	xong bước 2
RUN	4	0	10415	12592	−4	xong bước 3
RUN	5	0	11202	11942	−1	xong bước 4
RUN	6	1	11575	11592	0	xong bước 5
RUN	7	0	11394	11772	−1	xong bước 6
SAMPLE	7	0	11485	11683	−1	xong bước 7
IDLE	7	0	11485	11683	−1	done=1, sin 91, cos 89

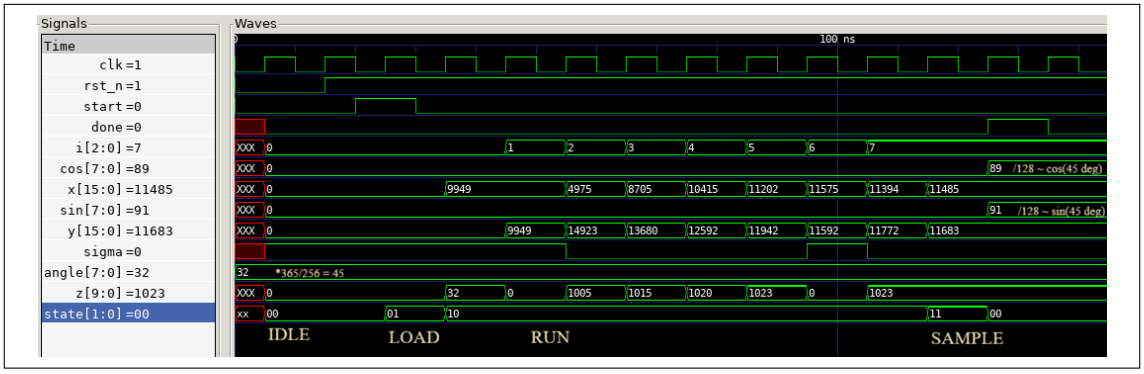
Ở nhịp done, sau trạng thái SAMPLE, sin và cos là x và y dịch phải 7 bit. 11683/128 = 91,27 và sin làm tròn thành 91. 11485/128 = 89,73 và cos làm tròn thành 89.

Giá trị thật của 45° là sin = cos = 0,70710678. Cùng thang 128 thì

$$128 \times 0{,}70710678 = 90{,}51,$$

và 90,51 làm tròn thành mã 91. sin trên ảnh khớp mã này. cos lệch 2 mã, và lệch đó là sai số của tám bước.

Trên ảnh, con trỏ đứng ở nhịp done. state là 00, i là 7, x là 11485 và y là 11683. sin là 91 và cos là 89. z hiện 1023 vì cửa sổ vẽ số không dấu. Cùng 10 bit đó, đọc có dấu, là −1, đúng z ở hàng trạng thái SAMPLE.



Hình 5: GTKWave của mã 32, tại nhịp done.

3.5 Ca từng đỏ

Hai testbench (TB) đã lưu đều in dòng đạt (PASS) trên RTL gốc. Dòng của lõi là IO5 RTL TB PASS. Dòng của vỏ chân (PIN SHELL) là IO5 PIN SHELL PASS. Phép phá mạch dùng bản sao của cordic_control.v và bỏ dấu cosine: cos_cap bằng x8, không còn sign_cos ? -x8 : x8. Sáu cặp và dải ±4 của tb_cordic.v giữ nguyên. Mã 0 và mã 64

còn trong dải. Mã 128 làm cosine thành 127, trong khi mong đợi là −128, và bench dừng bằng `$fatal`. Ba mã sau, 192, 32 và 224, vẫn trong dải khi chạy tiếp cùng luật. File gốc không đổi.

Bảng 3.1 — Ca kiểm thử và kết quả

#	Ca kiểm thử	Đầu vào	Mong đợi	Thu được	Đạt	Vì sao chọn ca này
1	0°	0	0, 127	−2, 127	có	đầu dải
2	gần 30°	21	63, 111	64, 110	có	mã gần 30°
3	45°	32	91, 91	91, 89	có	đúng mã
4	gần 60°	43	111, 63	113, 59	có	mã gần 60°
5	90°	64	127, 0	127, 4	có	biên sai số cosine bằng 4
6	135°, tự chọn	96	91, −91	91, −89	có	cùng `z_load` với 45°, đảo `sign_cos`
7	180°	128	0, −128	2, −127	có	phần tư thứ ba
8	270°	192	−128, 0	−127, −4	có	phần tư thứ tư
9	315°	224	−91, 91	−91, 89	có	đối của 45°
10	biên, tự chọn	255	−3, 127	−4, 127	có	một bước trước đủ vòng

Trong Bảng 3.1, mỗi ô ghi sine trước rồi cosine sau. Bench sáu cặp chấp nhận lệch trong ±4. Trên mười mã này, lệch sine lớn nhất là 2 và lệch cosine lớn nhất là 4.

Bảng 3.2 — Phép thử phá mạch (bắt buộc)

Lỗi cố tình tạo ra	Số ca chuyển sang đỏ	Ca nào đỏ	Kết luận về chất lượng testbench
bỏ `sign_cos` ở `cos_cap`	1 / 6	mã 128	bench dừng khi cosine thành 127 thay vì −128

4 Kết quả tổng hợp

4.1 Ước lượng trước khi đối chiếu

Số flip-flop được đếm từ RTL trước khi map, và tổng là 66 bit. Map sky130 của lần chạy `RUN_2026.10.03_09.21.28` có 66 cell `dfrtp_1`. Bảng 4.1 đặt ước lượng cạnh số thật.

4.2 Đối chiếu flip-flop

66 bit khai báo gồm `x` và `y` 16 bit mỗi thanh ghi, cùng `z` 10 bit. `sin` và `cos` chiếm 8 bit mỗi thanh ghi. `i` chiếm 3 bit, `state` chiếm 2 bit, `done` chiếm 1 bit, và `sign_sin` với `sign_cos` chiếm phần còn lại. Mỗi bit thành một cell `dfrtp`: một flip-flop lấy dữ liệu ở cạnh lên của `clk` và xóa về 0 khi `rst_n` thấp. Hậu tố `_1` hay `_2` là sức kéo của cổng ra, không phải số bit. Map ra đúng 66 flip-flop. Lệch bằng 0, nên không có thanh ghi thừa và cũng không mất thanh ghi.

4.3 Loại cell

File `1-synthesis.AREA_0.stat.rpt` ghi diện tích lõi lúc map là 7553,49 μm^2 và 672 cell. File đó đếm từng tên cell, không nhân số cell với diện tích một cell, nên chưa chỉ được loại nào chiếm nhiều μm^2 nhất. Tên lặp nhiều nhất trong file này là flip-flop, 66 lần. Bản đếm 734 cell ở bước Yosys trước flow có 22 cell `maj3_1`. Cổng `maj3` ra mức 1 khi ít nhất hai trong ba đầu vào bằng 1. Yosys chọn cổng này lúc map phép cộng và phần logic còn lại. Không có cell nhân và không có bộ nhớ. Bảng 4.2 tách bốn loại cell đếm được.

Bảng 4.1 — Số liệu tổng hợp

Chỉ số	Ước lượng trước khi chạy	Kết quả thật	Chênh lệch và giải thích
Tổng số standard cell	không ước lượng trước	734	số sau map sky130
Số flip-flop	66 bit từ RTL	66 `dfrtp_1`	lệch 0
Diện tích cell (μm^2)	—	7553,49	tổng lúc map, chưa tách theo loại
Hệ số sử dụng (%)	45, mục tiêu đặt	23,41 lúc đặt	`Final_Util` $= -1$, chưa có số sau đi dây

Bốn số cell là bốn bước khác nhau, không phải bốn lần đếm cùng một tập. 734 là số cell sau map sky130 ở bước Yosys trước flow. 672 là số cell trong `1-synthesis.AREA_0.stat.rpt` của chính lần chạy, và ở file đó flip-flop mang tên `dfrtp_2`, vẫn 66 lần. 755 là số cell đã

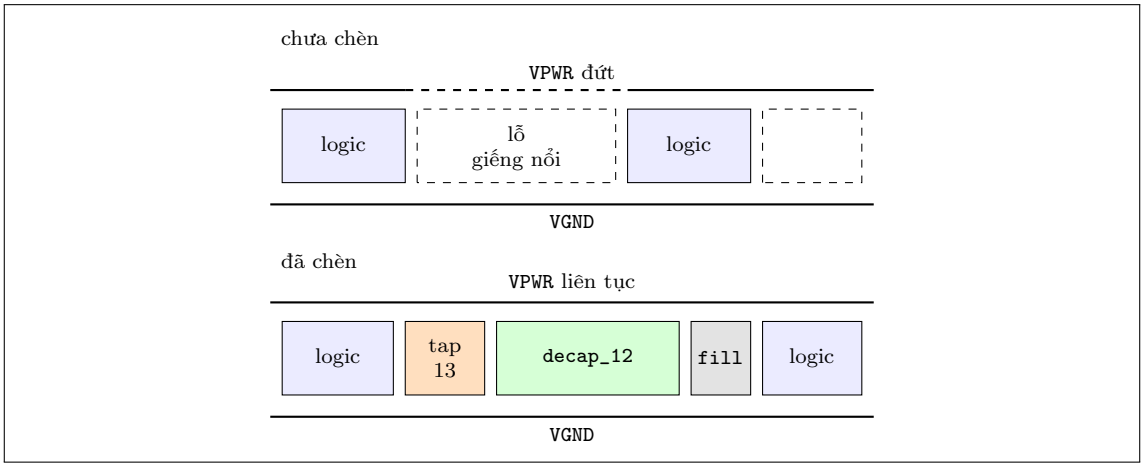
đặt lên đế, không kể filler. 3901 là 755 cộng 3146 cell không tính toán.

Một hàng standard cell có hai ray. `VPWR` là ray nguồn. `VGND` là ray đất. Cell logic, gồm `dfrtp` và `maj3`, nằm giữa hai ray và làm tính toán. Ba cell còn lại không tính `sin` hay `cos`. `config.tcl` của thiết kế không bật hai công tắc này. OpenLane ghi mặc định `RUN_TAP_DECAP_INSERTION` và `RUN_FILL_INSERTION` bằng 1 vào `config.tcl` của lần chạy, nên công cụ tự chèn chúng sau khi đặt cell logic. Hình 6 đối chiếu một hàng còn lỗ với hàng đã chèn. Đó là sơ đồ, không phải ảnh cắt từ file GDSII.

Tap, tên `tapvpwrvgnd_1`, nối giếng n với `VPWR` và đế p với `VGND`. Thiếu tap thì giếng nổi, transistor dễ latch-up, và luật khoảng cách giếng của sky130 bị phạm. Cách đặt thông thường là một lưới, không phải một tap cho mỗi cổng. File `config.tcl` của thiết kế không ghi khoảng này. OpenLane lấy mặc định của sky130 và chép vào `config.tcl` của lần chạy: `FP_TAPCELL_DIST` bằng 13, tức một tap cách tap kế khoảng 13 μm. Netlist cuối có 469 tap.

Decap là tụ giữa hai ray. Khi nhiều cổng đổi mức cùng lúc, tụ này cấp điện tích tại chỗ, nên điện áp trên ray sụt ít hơn. Thiếu decap thì sụt áp trên ray tăng. Công cụ không vẽ tay từng tụ. Biến `DECAP_CELL` liệt kê từ rộng đến hẹp: `decap_12`, `decap_8`, `decap_6`, `decap_4`, rồi `decap_3`. Chỗ trống nhận bản rộng nhất còn vừa. Vì vậy `decap_12` chiếm 1239 trong 2103 decap.

Filler, tên `fill_1` và `fill_2`, không có chân logic và gần như không có điện dung. Nó chỉ bịt ô rộng 1 hoặc 2 mà decap không vừa, để ray và giếng không đứt đoạn. Bỏ filler thì DRC mật độ và tính liên tục của ray hỏng, dù chức năng RTL không đổi. Netlist cuối có 574 filler, gồm `fill_1` 432 và `fill_2` 142. Không cell nào trong ba nhóm này giữ `x`, `y` hay `z`.



Hình 6: Cách chèn trên một hàng. Hàng trên còn lỗ: giếng chưa được tap buộc, chỗ trống làm ray đứt. Hàng dưới là thứ tự của lần chạy: tap theo lưới 13 μm, decap rộng nhất còn vừa, filler bịt ô còn lại. Đây không phải ảnh của đế `cordic`.

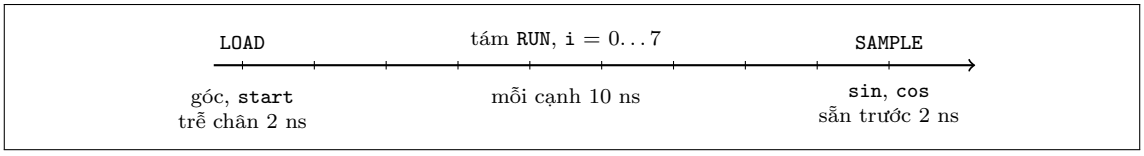
4.4 Những gì thiết kế đặt vào chuỗi nhịp

Chuỗi một lần tính do RTL đặt: một nhịp `LOAD`, tám nhịp `RUN` và một nhịp `SAMPLE`. Đó là mười cạnh `clk`. Hình 7 gắn mỗi cạnh với dòng đã ghi trong file thiết kế. Tap, decap và filler không nằm trên chuỗi này.

`CLOCK_PERIOD` bằng 10 ns là thời gian cho phép của từng cạnh, không phải của cả mười cạnh. Đường từ một thanh ghi tới thanh ghi kế phải xong trong 10 ns đó. Slack setup 4,73 ns là phần còn dư bên trong một cạnh, không phải dư của tám bước `RUN`. Mười cạnh đặt cạnh nhau là $10 \times 10\,\text{ns} = 100\,\text{ns}$ theo ràng buộc. Đó không phải thời gian đo trên chip.

`IO_PCT` bằng 0,2 dành 2 ns ở biên của cùng cạnh đó. Ở cạnh vào `LOAD`, mã góc và `start` được coi là đã tới chân sau 2 ns, nên phần trong lõi còn 8 ns. Ở cạnh ra khỏi `SAMPLE`, `sin` và `cos` phải sẵn sàng trước 2 ns cho mạch ngoài. `SYNTH_ADDER_TYPE` bằng `YOSYS` giữ tám nhịp `RUN` là cộng, trừ và dịch. Vì vậy bảng cell không có bộ nhân. `SYNTH_STRATEGY` bằng `AREA 0` chỉ bảo Yosys map ít cell hơn. Nó không đổi số nhịp.

Đế 200 μm và mật độ 0,45 không đổi chuỗi. Hai dòng đó chỉ chừa chỗ cho cùng 66 thanh ghi. Khoảng tap, danh sách decap và hai công tắc filler không có trong `config.tcl` của thiết kế. Flow không dừng giữa chừng để sửa một filler hay một slack. Muốn chuỗi ngắn hơn thì sửa RTL. Muốn mỗi cạnh ngắn hơn thì sửa `CLOCK_PERIOD` rồi chạy lại. Lần chạy đã nộp không làm cả hai.



Hình 7: Mười cạnh của một lần tính. Mỗi cạnh là một `CLOCK_PERIOD` bằng 10 ns do `config.tcl` đặt. Trễ chân 2 ns nằm ở cạnh `LOAD` và ở cạnh ra khỏi `SAMPLE`.

Bảng 4.2 — Phân loại cell

Loại cell	Số lượng	Vai trò trong mạch
`dfrtp_1`	66	thanh ghi của `x`, `y`, `z`, trạng thái và kết quả
`maj3_1`	22	cổng majority sau map, không phải bộ nhân
bộ nhân	0	tám bước không nhân
bộ nhớ	0	không có bảng sine

5 Kết quả phân tích thời gian

5.1 Setup và hold

Ràng buộc chu kỳ là 10 ns và trễ chân là 2 ns. Số thời gian dưới đây lấy từ bước signoff, tức bước kiểm sau khi dây đã đi. File là `31-rcx_sta`. Chữ rcx là phép rút điện trở và tụ của chính những dây đó. Static timing analysis (STA) trên file này dùng điện trở và tụ đã rút, không dùng độ trễ ước lượng trước khi đi dây.

Worst negative slack (WNS) và total negative slack (TNS) trong `metrics.csv` đều bằng 0. WNS bằng 0 chỉ có nghĩa không có đường nào trễ quá chu kỳ. Nó không phải độ dư của đường chậm nhất. Trong `31-rcx_sta.summary.rpt`, slack setup lớn nhất là 4,73 ns và được ghi MET, tức còn trong hạn. Slack hold nhỏ nhất là 0,29 ns, cũng MET. Hold là đường ngắn nhất: dữ liệu không được tới chân D quá sớm so với cùng cạnh `clk`.

Slack của một đường là hạn cho phép trừ thời điểm dữ liệu tới. Trên đường setup chậm nhất, hạn là 10,08 ns và dữ liệu tới lúc 5,35 ns, nên slack bằng 4,73 ns. Trên đường hold ngắn nhất, dữ liệu tới lúc 0,92 ns và hạn là 0,63 ns, nên slack bằng 0,29 ns. Số dương nghĩa là đường còn trong hạn. WNS bằng 0 vì công cụ chỉ ghi phần âm, và ở đây không có phần âm.

5.2 Đường tới hạn

Đường setup chậm nhất trong `31-rcx_sta.max.rpt` đi từ flip-flop `_1240_` đến flip-flop `_1226_`. Cổng Q của `_1240_` là bit `u_ctrl.i[0]`, tức bit 0 của bộ đếm vòng trong khối điều khiển. Cổng Q của `_1226_` mang tên net `u_ctrl.u_sin.t[10]`, một dây bên trong khối thu hẹp `y` thành `sin`. Dữ liệu tới chân D lúc 5,35 ns. Hạn cho phép là 10,08 ns. Hiệu hai số là slack 4,73 ns.

Từ Q đến D có 15 cell tổ hợp: một đệm, `mux4`, `mux2`, `or4`, một đệm nữa, `o31a`, hai `xor2`, `and2b`, `a311o`, `a211oi`, `nor2`, `o211a`, `or3` và `a32o`. Đường này nằm trong khối điều khiển, không phải bộ cộng 16 bit của `x` và `y`.

Đường hold ngắn nhất trong `31-rcx_sta.min.rpt` xuất phát và kết thúc cùng flip-flop `_1213_`. Dây ra là `u_ctrl.u_sin.v[2]`. Dữ liệu tới sau 0,92 ns, hạn hold là 0,63 ns, slack 0,29 ns.

5.3 Tần số ước tính

100 MHz là 1/10 ns, tức tần số của ràng buộc. Vì slack setup còn 4,73 ns, một ước lượng chu kỳ ngắn hơn là $10 - 4{,}73 = 5{,}27$ ns, khoảng 190 MHz. Công cụ vẫn ghi tần số gợi ý

100 MHz, vì nó chỉ hạ gợi ý khi WNS âm. Cả hai số đều là mô hình STA, không phải tần số đo trên chip. Bảng 5.1 chép chu kỳ, slack setup, slack hold và tần số của ràng buộc.

5.4 Mạch có thanh ghi

Mạch có thanh ghi, nên STA có đường từ thanh ghi này đến thanh ghi kia. Slack setup 4,73 ns và slack hold 0,29 ns đều thuộc các đường đó, dưới chu kỳ 10 ns. Mạch này không phải mạch tổ hợp thuần.

5.5 Không hạ chu kỳ

WNS không âm dưới chu kỳ 10 ns. Lần chạy đã giữ vẫn là layout duy nhất được trích. Không có lần chạy thứ hai với chu kỳ dài hơn.

Bảng 5.1 — Số liệu thời gian

Chỉ số	Giá trị	Đạt / Vi phạm	Nhận xét
Chu kỳ ràng buộc (ns)	10	—	trễ chân 2 ns
Worst setup slack (ns)	4,73	đạt, MET	WNS ghi 0 vì không có slack âm
Worst hold slack (ns)	0,29	đạt, MET	`31-rcx_sta.min.rpt`
Tần số của ràng buộc (MHz)	100	—	$f = 1/10$ ns, không phải tần số đo

Đệm nhịp `clkbuf_3_2__f_clk` kéo 11 tải, trong khi giới hạn của lần chạy là 10. Cảnh báo này không làm WNS thành âm.

Bảng 5.2 — Đường tới hạn

Hạng mục	Nội dung
Điểm xuất phát	`_1240_`, Q là `u_ctrl.i[0]`
Điểm kết thúc	`_1226_`, Q là `u_ctrl.u_sin.t[10]`
Đi qua các khối	15 cell tổ hợp trong khối điều khiển, từ bit `i` tới đường thu `sin`
Số tầng logic	15 cell từ Q đến D
Vì sao chính đường này chậm nhất	dữ liệu tới lúc 5,35 ns, trễ hơn mọi đường setup khác trong file
Một cách cụ thể để rút ngắn nó	cắt bớt tầng mux và xor trên đường từ `i[0]` tới thanh ghi `sin`

6 Công suất và kiểm tra vật lý

6.1 Công suất

`31-rcx_sta.power.rpt` ước lượng công suất ở góc điển hình, sau khi đã rút điện trở và tụ của dây. Công suất động gồm hai phần: internal $2{,}37 \times 10^{-3}$ W, tức hao bên trong cell khi đổi mức, và switching $2{,}23 \times 10^{-3}$ W, tức hao để nạp tụ dây. Công suất tĩnh, tức dòng rò, là $6{,}07 \times 10^{-9}$ W. Tổng là $4{,}59 \times 10^{-3}$ W. Internal chiếm 51,5% và switching chiếm 48,5%. Rò gần như bằng 0 trong bảng này. Nhóm tổ hợp chiếm 85,8% tổng công suất, nhóm thanh ghi chiếm 14,2%. Đây là ước lượng của mô hình, không phải số đo trên chip.

6.2 Luật vẽ và đối chiếu sơ đồ

Ba phép kiểm hình học của cùng lần chạy đều có số. Magic báo design rule check (DRC) bằng 0, layout versus schematic (LVS) sạch, và kiểm ăng-ten bằng 0. DRC chỉ hỏi hình vẽ có phạm luật vẽ hay không. LVS chỉ hỏi layout có khớp sơ đồ nối hay không. Cả hai phép không đọc sine và cosine.

6.3 Luật vẽ không chứng minh chức năng

DRC bằng 0 không có nghĩa mạch tính đúng sine và cosine. Phép này chỉ đếm vi phạm hình học. Chức năng nằm ở Bảng 3.1. Trên mười mã của bảng đó, lệch cosine lớn nhất là 4 mã. Ảnh đế của cùng lần chạy nằm ở mục sau.

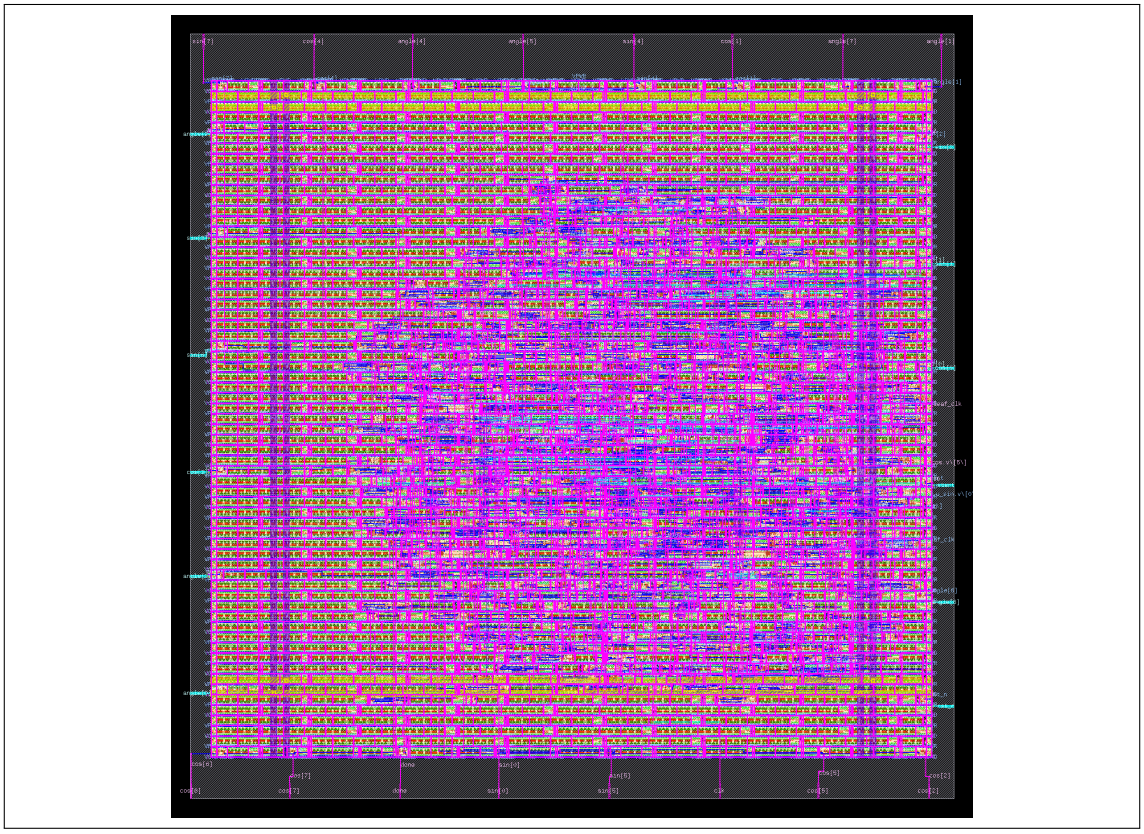
Bảng 6.1 — Công suất và kiểm tra vật lý

Hạng mục	Giá trị / Kết quả	Nhận xét và giới hạn
Công suất động	$4{,}60 \times 10^{-3}$ W	internal $2{,}37 \times 10^{-3}$ cộng switching $2{,}23 \times 10^{-3}$
Công suất tĩnh (rò)	$6{,}07 \times 10^{-9}$ W	rất nhỏ so với phần động
Tổng công suất	$4{,}59 \times 10^{-3}$ W	góc điển hình, file `31-rcx_sta.power.rpt`
DRC	0 vi phạm	luật hình học, không phải chức năng
LVS	sạch	layout khớp sơ đồ, không phải bảng sai số
ăng-ten	0 vi phạm	luật ăng-ten của dây, không phải chức năng

7 Quan sát layout

7.1 Ảnh layout

Hình 8 là ảnh hai chiều của file GDSII, lõi `cordic`, lần chạy đã giữ. Đỉnh của file là `cordic`. Vỏ `tt_um_cordic` chỉ được mô phỏng và không có trong file này.



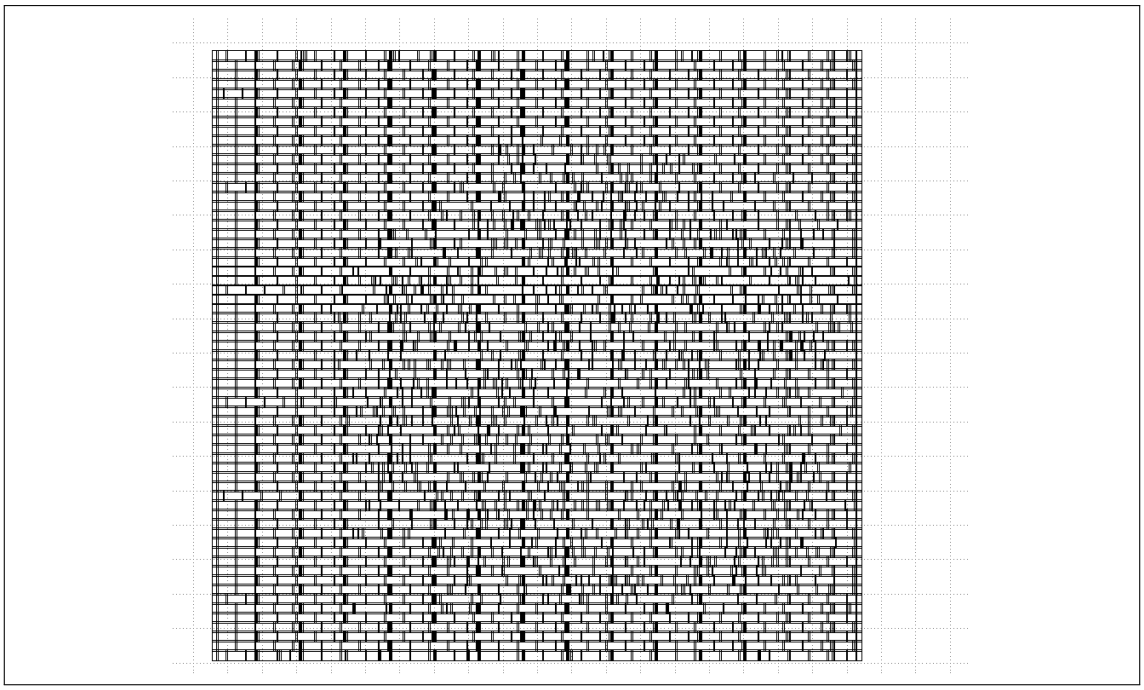
Hình 8: Ảnh hai chiều của file GDSII, lõi `cordic`: hàng cell và dây trên đế 200 μm, lần chạy `RUN_2026.10.03_09.21.28`.

Hình 9 là cùng đế khi chỉ còn các hàng. Vệt ngang đều nhau ở giữa là hàng standard cell. Bước quy trình tương ứng là đặt cell.

Hình 10 là cùng đế với hai lớp kim loại đang bật. Dây xanh đi ngang. Dây tím đi dọc. Hình 11 là cửa sổ KLayout của chính file đó: lớp đang chọn cho thấy `met1.drawing` (68/20) là xanh và `met2.drawing` (69/20) là tím. Vậy trên đế này `met1` đi ngang và `met2` đi dọc. Via là ô nhỏ nơi một dây đổi giữa hai lớp. Số 5571 là số via của cả đế trong báo cáo lần chạy, không phải số đếm bằng mắt trên ảnh.

7.2 Những gì nhìn thấy

Đế là hình vuông cạnh 200 μm. Vùng giữa ảnh là các hàng cell, như Hình 9. Dây đã đi dài 23073 đơn vị của công cụ, với 5571 via. Hướng của hai lớp dưới đọc từ Hình 10 và



Hình 9: Hàng standard cell trên `cordic.gds`. Vệt ngang ở giữa đế là hàng cell của bước đặt.

bảng lớp ở Hình 11: `met1` ngang, `met2` dọc. Chân trên đế là chân của lõi.

Phần lớn diện tích hàng không phải cổng của CORDIC. 755 cell logic nằm xen với 3146 filler, tap và decap. Nhìn trên ảnh, những mảnh đều trong hàng là các cell lấp chỗ. Cổng tính `sin` và `cos` là phần còn lại của cùng những hàng đó. Via là chỗ một dây đổi lớp kim loại. 5571 via là số lỗ như vậy trên cả đế, không phải số cổng.

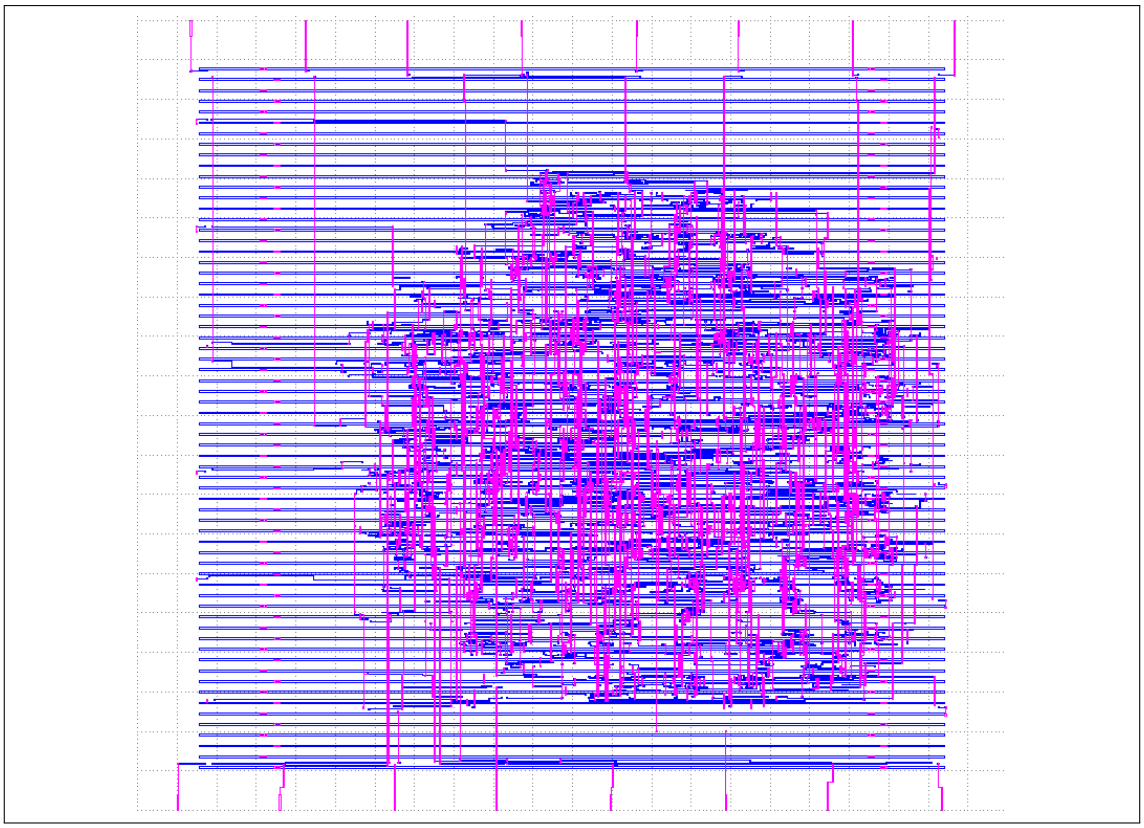
7.3 Liên hệ quy trình

Hàng cell trên ảnh là chỗ công cụ đặt transistor và lỗ tiếp xúc, nhìn ở mức ảnh layout. Dây và via là các lớp kim loại xếp lên nhau. Nhận xét này đọc từ file GDSII. Nó không phải số đo của một mẻ chế tạo.

7.4 Phương pháp của lần chạy

Đi dây có hai cách thông dụng. Một cách là đi chi tiết ngay trên toàn đế. Cách kia là đi dây toàn cục trước, để chừa đường, rồi đi dây chi tiết. Lần chạy này dùng cách thứ hai của OpenLane. Kết quả của cách đó trên đế này là chiều dài dây 23073 đơn vị và 5571 via.

Nhịp cũng có hai cách. Sao là một đệm nối thẳng tới mọi flip-flop. Cây là đệm gốc rồi đệm lá. Lần chạy này dùng cây. Trên đường setup, `clk` vào `clkbuf_0_clk`, đệm gốc loại `clkbuf_16`, fanout 8 trên `clknet_0_clk`. Đệm lá `clkbuf_3_6__f_clk` kéo 7 flip-flop.



Hình 10: Hai lớp kim loại trên `cordic.gds`. Xanh là `met1`, đi ngang. Tím là `met2`, đi dọc.

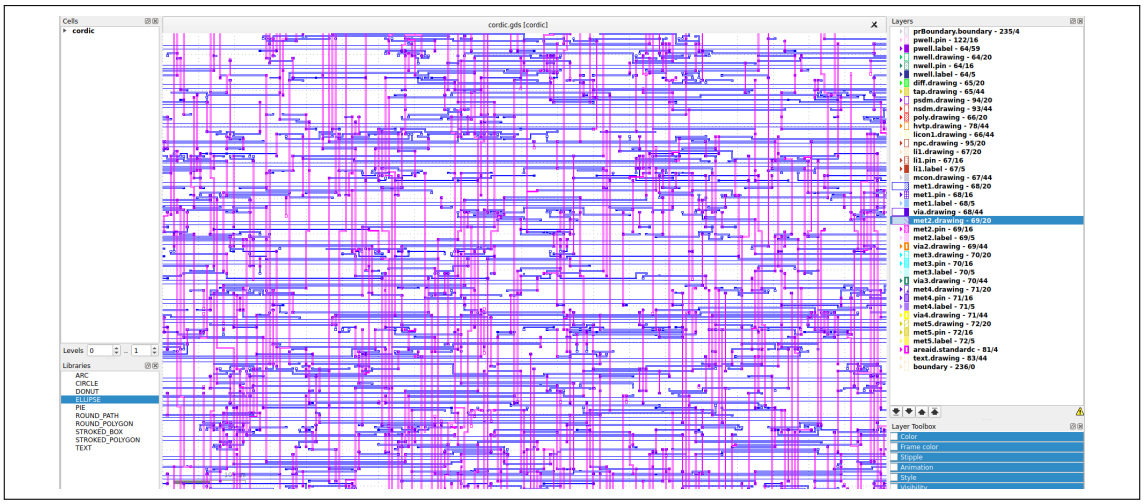
Một đệm lá khác, `clkbuf_3_2__f_clk`, kéo 11 tải trong khi giới hạn là 10.

Hàng cell có thể chèn tap, decap và filler bằng tay, hoặc để flow chèn sau khi đặt cell logic. Lần chạy này để flow chèn, vì `config.tcl` của thiết kế không ghi lưới tap. Mặc định sky130 được chép vào lần chạy: tap cách khoảng 13 μm, decap lấy bản rộng nhất còn vừa, filler bịt ô 1 và ô 2. Netlist cuối có 469 tap, 2103 decap và 574 filler.

Thời gian có thể ước lượng trước khi có dây, hoặc rút điện trở và tụ của dây đã đi rồi mới phân tích. Lần chạy này dùng cách sau. File `31-rcx_sta` là phân tích đó: slack setup 4,73 ns, slack hold 0,29 ns.

Bảng 7.1 gắn những gì nhìn thấy trên các ảnh với bước đặt cell và bước đi dây. Đệm nhịp không được khoanh trên các ảnh này. Tên của nó nằm trong netlist.

Bảng 7.1 — Quan sát layout



Hình 11: Cửa sổ KLayout của `cordic.gds`, đỉnh `cordic`. Bảng lớp ghi `met1.drawing` màu xanh và `met2.drawing` màu tím.

Nhìn thấy gì	Ở đâu trên hình	Liên hệ với bước quy trình nào
hàng standard cell	vệt ngang giữa Hình 9	đặt cell lên đế
`met1` đi ngang, `met2` đi dọc	Hình 10, tên lớp ở Hình 11	các lớp kim loại xếp chồng
5571 via	số của cả đế, không đếm trên ảnh	lỗ đổi lớp kim loại
đệm nhịp 11 tải	netlist, không khoanh trên ảnh	phân phối xung, chỉ có ở mạch tuần tự

8 Phần đóng góp của AI

AI được dùng như một trợ lý khi viết báo cáo. AI không chạy lại flow và không tạo số liệu mới.

AI hỗ trợ trong việc chạy command, comment cho hình và kết quả chạy ra. AI gợi giải thích các bước chạy và gợi ý các tài nguyên để tìm hiểu sâu hơn về các lần chạy và kết quả.

9 Kết luận, giới hạn và tự đánh giá

9.1 Điều kiện của kết luận

Mọi số layout ở trên đến từ một lần chạy. Ảnh OpenLane là `ff5509f65b17` và process design kit (PDK) là `sky130A`. Chu kỳ ràng buộc là 10 ns và mật độ đặt mục tiêu là 0,45. `metrics.csv` ghi `OpenDP_Util` bằng 23,41, tức mật độ lúc đặt cell, và ghi `Final_Util` bằng -1. Giá trị -1 nghĩa là flow không lưu hệ số sử dụng sau khi đi dây. Diện tích lõi là 33344,48 μm^2 trên đế 200 μm $\times$ 200 μm.

9.2 Việc chưa có số

Diện tích tách theo từng loại cell không có: file thống kê chỉ cho tổng diện tích lõi lúc tổng hợp, 7553,49 μm^2, với 672 cell, trong đó `dfrtp` xuất hiện 66 lần. Kho không có Actions. Log phép phá mạch nằm ở Bảng 3.2, ngày 26/10/2026. Bảng 9.1 chỉ kết luận từ số đã có.

9.3 Điều không được kết luận

Bảng 9.1 ghi mỗi kết luận cạnh điều kết luận đó không chứng minh. DRC bằng 0 không chứng minh chức năng. 100 MHz không phải tần số đo trên chip. Mỗi hàng của bảng nhắc điều kiện đã tạo ra số đó.

9.4 Chip không được chế tạo

Lần chạy là layout cục bộ của lõi `cordic`. Chip không được chế tạo. Không có số nào trong báo cáo này là số đo trên silicon. Các kết luận ở Bảng 9.1 dừng ở mô phỏng và ở file layout.

Bảng 9.1 — Kết luận có điều kiện

Kết luận	Điều kiện đã tạo ra nó	Điều này không chứng minh
tám bước, sai số sine tối đa 2, cosine tối đa 4	mười mã, công thức 128 sin θ, ngày 2026-10-04	không phải mọi mã 0–255
734 cell, 66 flip-flop, 0 nhân	map sky130, PDK `sky130A`	không phải diện tích đã tiết kiệm so với bộ nhân
`wns` = 0 ở 10 ns	dây đã đi, trễ chân 2 ns	không phải tần số đo trên chip
DRC 0, LVS sạch	Magic, cùng lần chạy	không phải chức năng đúng

Bảng 9.2 ghi mức Khá cho đặc tả, tổng hợp và thời gian.

Bảng 9.2 — Tự đánh giá

Tiêu chí	Mức	Lý do
1. Đặc tả và sơ đồ khối	Khá	bốn trạng thái, tám giả định, ba slide `top`, `pu`, `cu`
3. Đọc báo cáo tổng hợp	Khá	66 khớp 66, có tổng diện tích 7553,49 μm^2, chưa tách theo loại cell
4. Đọc báo cáo thời gian	Khá	slack setup 4,73 ns, hold 0,29 ns, có tên hai đầu đường

A Mã nguồn RTL đầy đủ

cordic.v — lõi cordic: u_proc cạnh u_ctrl

```
‘default_nettype none
‘timescale 1ns/1ps

// Top: control unit beside the processing unit.
// The processing unit contains Block B (z, sigma) and Block A (x, y).
module cordic (
    input  wire              clk,
    input  wire              rst_n,
    input  wire              start,
    input  wire       [7:0]  angle,
    output wire              done,
    output wire signed [7:0] sin,
    output wire signed [7:0] cos
);

    wire               load;
    wire               run;
    wire        [2:0]  i;
    wire signed [15:0] x, y;
    wire               sign_sin, sign_cos;

    cordic_control u_ctrl (
        .clk   (clk),
        .rst_n (rst_n),
        .start (start),
        .x        (x),
        .y        (y),
        .sign_sin (sign_sin),
        .sign_cos (sign_cos),
        .load     (load),
        .run   (run),
        .i     (i),
        .done  (done),
        .sin   (sin),
        .cos   (cos)
```

```
    );

    cordic_processing u_proc (
        .clk   (clk),
        .rst_n (rst_n),
        .load  (load),
        .run   (run),
        .i     (i),
        .angle (angle),
        .x        (x),
        .y        (y),
        .sign_sin (sign_sin),
        .sign_cos (sign_cos)
    );

endmodule
```

tt_um_cordic.v — vỏ chân Tiny Tapeout, nối ui_in và uio_in[0] vào lõi

```
‘default_nettype none
‘timescale 1ns/1ps

// Tiny Tapeout shell. Charter pins, which his package named and did not draw.
//   ui_in[7:0]  angle
//   uio_in[0]   start
//   uo_out[7:0] sin
//   uio_out[7:0] cos
// clk and rst_n are the chip pins beside those 24.
// uio_in and uio_out are the same eight wires. They are inputs while the
// core is waiting, and they drive cos on the one cycle done is high.
// start is ignored on that cycle so cos[0] cannot look like a new start.
module tt_um_cordic (
    input  wire [7:0] ui_in,
    output wire [7:0] uo_out,
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
```

```
    input  wire       rst_n
);

    wire              done;
    wire signed [7:0] sin;
    wire signed [7:0] cos;

    cordic u_core (
        .clk   (clk),
        .rst_n (rst_n),
        .start (uio_in[0] & ~done),
        .angle (ui_in),
        .done  (done),
        .sin   (sin),
        .cos   (cos)
    );

    assign uo_out  = sin;
    assign uio_out = cos;
    assign uio_oe  = done ? 8'hff : 8'h00;

    wire _unused = &{ena, uio_in[7:1], 1'b0};

endmodule
```

cordic_control.v — khối điều khiển trên slide cu

```
`default_nettype none
`timescale 1ns/1ps

// Control unit. State register, iteration register, and the output sample.
module cordic_control (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               start,
    input  wire signed [15:0] x,
    input  wire signed [15:0] y,
    input  wire               sign_sin,
    input  wire               sign_cos,
```

```
    output wire               load,
    output wire               run,
    output wire        [2:0]  i,
    output wire               done,
    output wire signed [7:0]  sin,
    output wire signed [7:0]  cos
);
    localparam LOAD = 2'd1;
    localparam RUN  = 2'd2;

    wire [1:0] state;
    wire [1:0] state_next;
    wire [2:0] i_next;
    wire       sample;
    wire signed [7:0] x8, y8;
    wire signed [7:0] sin_cap, cos_cap;
    wire [7:0] sin_d, cos_d;
    wire       done_next;

    ctrl_next u_next (
        .state      (state),
        .i          (i),
        .start      (start),
        .state_next (state_next),
        .i_next     (i_next),
        .sample     (sample)
    );

    dreg #(.WIDTH(2)) u_state (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (state_next),
        .q     (state)
    );

    dreg #(.WIDTH(3)) u_i (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (i_next),
```

```
    .q     (i)
);

assign load = (state == LOAD);
assign run  = (state == RUN);

narrow_i8 u_cos (
    .v  (x),
    .y8 (x8)
);

narrow_i8 u_sin (
    .v  (y),
    .y8 (y8)
);

assign cos_cap = sign_cos ? -x8 : x8;
assign sin_cap = sign_sin ? -y8 : y8;

sample_mux u_cos_mux (
    .sample   (sample),
    .hold     (cos),
    .captured (cos_cap),
    .d        (cos_d)
);

sample_mux u_sin_mux (
    .sample   (sample),
    .hold     (sin),
    .captured (sin_cap),
    .d        (sin_d)
);

dreg #(.WIDTH(8)) u_cos_reg (
    .clk   (clk),
    .rst_n (rst_n),
    .d     (cos_d),
    .q     (cos)
);
```

```
    dreg #(.WIDTH(8)) u_sin_reg (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (sin_d),
        .q     (sin)
    );

    assign done_next = sample;
    dreg #(.WIDTH(1)) u_done (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (done_next),
        .q     (done)
    );
endmodule
```

cordic_processing.v — khối xử lý trên slide pu

```
‘default_nettype none
‘timescale 1ns/1ps

// Processing unit. Block B produces sigma. Block A rotates x and y.
// load writes the starting values. run updates one iteration.
module cordic_processing (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               load,
    input  wire               run,
    input  wire        [2:0]  i,
    input  wire        [7:0]  angle,
    output wire signed [15:0] x,
    output wire signed [15:0] y,
    output wire               sign_sin,
    output wire               sign_cos
);

    localparam signed [15:0] K_XY = 16’sd9949;  // 0.607253 * 16384
```

```
wire              sigma;
wire signed [9:0] z;
wire signed [9:0] z_load;
wire              sign_sin_now, sign_cos_now;
wire              sign_sin_d, sign_cos_d;

angle_fold u_angle (
    .angle    (angle),
    .z_load   (z_load),
    .sign_sin (sign_sin_now),
    .sign_cos (sign_cos_now)
);

// Captured with z, so a later change of angle does not flip the quadrant.
assign sign_sin_d = load ? sign_sin_now : sign_sin;
assign sign_cos_d = load ? sign_cos_now : sign_cos;
dreg #(.WIDTH(1)) u_sign_sin (
    .clk   (clk),
    .rst_n (rst_n),
    .d     (sign_sin_d),
    .q     (sign_sin)
);
dreg #(.WIDTH(1)) u_sign_cos (
    .clk   (clk),
    .rst_n (rst_n),
    .d     (sign_cos_d),
    .q     (sign_cos)
);

cordic_block_b u_b (
    .clk    (clk),
    .rst_n  (rst_n),
    .load   (load),
    .run    (run),
    .i      (i),
    .z_load (z_load),
    .z      (z),
    .sigma  (sigma)
);
```

```
    cordic_block_a u_a (
        .clk    (clk),
        .rst_n  (rst_n),
        .load   (load),
        .run    (run),
        .i      (i),
        .sigma  (sigma),
        .x_load (K_XY),
        .y_load (16'sd0),
        .x      (x),
        .y      (y)
    );

endmodule
```

cordic_block_a.v — u_a, cập nhật x và y

```
`default_nettype none
`timescale 1ns/1ps

// Block A: two shift blocks, two {load, run} muxes, two D registers.
module cordic_block_a (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               load,
    input  wire               run,
    input  wire        [2:0]  i,
    input  wire               sigma,
    input  wire signed [15:0] x_load,
    input  wire signed [15:0] y_load,
    output wire signed [15:0] x,
    output wire signed [15:0] y
);
    wire        [1:0]  sel = {load, run};
    wire signed [15:0] x_rot, y_rot;
    wire signed [15:0] x_d, y_d;

    shift_block u_x_rot (
```

```
    .main_axis    (x),
    .sub_axis     (y),
    .i            (i),
    .sigma        (sigma),
    .add_on_sigma (1'b0),
    .result       (x_rot)
);

shift_block u_y_rot (
    .main_axis    (y),
    .sub_axis     (x),
    .i            (i),
    .sigma        (sigma),
    .add_on_sigma (1'b1),
    .result       (y_rot)
);

axis_mux u_x_mux (
    .sel     (sel),
    .hold    (x),
    .on_load (x_load),
    .on_run  (x_rot),
    .d       (x_d)
);

axis_mux u_y_mux (
    .sel     (sel),
    .hold    (y),
    .on_load (y_load),
    .on_run  (y_rot),
    .d       (y_d)
);

axis_reg u_x (
    .clk   (clk),
    .rst_n (rst_n),
    .d     (x_d),
    .q     (x)
);
```

```
    axis_reg u_y (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (y_d),
        .q     (y)
    );
endmodule
```

cordic_block_b.v — u_b, cập nhật z

```
‘default_nettype none
‘timescale 1ns/1ps

// Block B: angle table, add/sub, {load, run} mux, z register, sigma.
module cordic_block_b (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               load,
    input  wire               run,
    input  wire        [2:0]  i,
    input  wire signed [9:0] z_load,
    output wire signed [9:0] z,
    output wire              sigma
);
    wire       [1:0] sel = {load, run};
    wire       [9:0] alpha_u;
    wire signed [9:0] alpha;
    wire signed [9:0] z_rot;
    wire signed [9:0] z_d;

    atan_lut u_lut (
        .i     (i),
        .alpha (alpha_u)
    );
    assign alpha = $signed(alpha_u);

    addsub_block #(.WIDTH(10)) u_z_rot (
        .main_axis    (z),
```

```
        .sub_axis     (alpha),
        .sigma        (sigma),
        .add_on_sigma (1'b0),
        .result       (z_rot)
    );

    axis_mux #(.WIDTH(10)) u_z_mux (
        .sel     (sel),
        .hold    (z),
        .on_load (z_load),
        .on_run  (z_rot),
        .d       (z_d)
    );

    axis_reg #(.WIDTH(10)) u_z (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (z_d),
        .q     (z)
    );

    sigma_block u_sigma (
        .z     (z),
        .sigma (sigma)
    );
endmodule

cordic_cells.v — ctrl_next, axis_mux, addsub_block, atan_lut, axis_reg

`default_nettype none
`timescale 1ns/1ps

// D flip-flop. The one leaf that stores a value.
module dreg #(
    parameter integer WIDTH = 1
) (
    input  wire                 clk,
    input  wire                 rst_n,
    input  wire [WIDTH-1:0]     d,
```

```
    output reg  [WIDTH-1:0]      q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= {WIDTH{1'b0}};
        else
            q <= d;
    end
endmodule

// Datapath register. Same cell as dreg, signed ports. x and y use 16. z uses 10.
module axis_reg #(
    parameter integer WIDTH = 16
) (
    input  wire                    clk,
    input  wire                    rst_n,
    input  wire signed [WIDTH-1:0] d,
    output wire signed [WIDTH-1:0] q
);
    dreg #(.WIDTH(WIDTH)) u_ff (
        .clk   (clk),
        .rst_n (rst_n),
        .d     (d),
        .q     (q)
    );
endmodule

// sel is {load, run}. load wins when both are 1.
module axis_mux #(
    parameter integer WIDTH = 16
) (
    input  wire        [1:0]       sel,
    input  wire signed [WIDTH-1:0] hold,
    input  wire signed [WIDTH-1:0] on_load,
    input  wire signed [WIDTH-1:0] on_run,
    output wire signed [WIDTH-1:0] d
);
    assign d = sel[1] ? on_load :
               sel[0] ? on_run  :
```

```
                        hold;
endmodule

// main +/- sub. add_on_sigma 1 adds when sigma is 1.
module addsub_block #(
    parameter integer WIDTH = 16
) (
    input  wire signed [WIDTH-1:0] main_axis,
    input  wire signed [WIDTH-1:0] sub_axis,
    input  wire                    sigma,
    input  wire                    add_on_sigma,
    output wire signed [WIDTH-1:0] result
);
    wire do_add = sigma ~^ add_on_sigma;
    assign result = do_add ? (main_axis + sub_axis) : (main_axis - sub_axis);
endmodule

// main +/- (sub >>> i). The x path uses add_on_sigma 0, the y path uses 1.
module shift_block (
    input  wire signed [15:0] main_axis,
    input  wire signed [15:0] sub_axis,
    input  wire        [2:0]  i,
    input  wire               sigma,
    input  wire               add_on_sigma,
    output wire signed [15:0] result
);
    wire signed [15:0] sub_shr = sub_axis >>> i;
    addsub_block u_as (
        .main_axis    (main_axis),
        .sub_axis     (sub_shr),
        .sigma        (sigma),
        .add_on_sigma (add_on_sigma),
        .result       (result)
    );
endmodule

// arctan(2^-i) in brads. 256 counts = 360 degrees. The last entry underflows to 0.
module atan_lut (
    input  wire       [2:0] i,
```

```
    output reg        [9:0] alpha
);
    always @* begin
        case (i)
            3'd0: alpha = 10'd32;
            3'd1: alpha = 10'd19;
            3'd2: alpha = 10'd10;
            3'd3: alpha = 10'd5;
            3'd4: alpha = 10'd3;
            3'd5: alpha = 10'd1;
            3'd6: alpha = 10'd1;
            default: alpha = 10'd0;
        endcase
    end
endmodule

// sigma = 1 when z is positive or zero.
module sigma_block (
    input  wire signed [9:0] z,
    output wire              sigma
);
    assign sigma = (z >= 0);
endmodule

// Fold one brad byte into the first quadrant and remember which quadrant it came fro
// 0..63, 64..127, 128..191, 192..255. z_load stays in 0..64.
module angle_fold (
    input  wire        [7:0] angle,
    output reg  signed [9:0] z_load,
    output reg               sign_sin,
    output reg               sign_cos
);
    always @* begin
        z_load   = 10'sd0;
        sign_cos = 1'b0;
        sign_sin = 1'b0;
        if (angle < 8'd64) begin
            z_load   = $signed({2'b00, angle});
            sign_cos = 1'b0;
```

```
            sign_sin = 1'b0;
        end else if (angle < 8'd128) begin
            z_load   = $signed({2'b00, (8'd128 - angle)});
            sign_cos = 1'b1;
            sign_sin = 1'b0;
        end else if (angle < 8'd192) begin
            z_load   = $signed({2'b00, (angle - 8'd128)});
            sign_cos = 1'b1;
            sign_sin = 1'b1;
        end else begin
            z_load   = $signed(10'd256 - {2'b00, angle});
            sign_cos = 1'b0;
            sign_sin = 1'b1;
        end
    end
endmodule

// 16-bit x,y with 1.0 at bit 14 -> 8-bit pin with 1.0 at bit 7.
module narrow_i8 (
    input  wire signed [15:0] v,
    output reg  signed [7:0]  y8
);
    reg signed [15:0] t;
    always @* begin
        t = v >>> 7;
        if (t > 16'sd127)
            y8 = 8'sd127;
        else if (t < -16'sd128)
            y8 = -8'sd128;
        else
            y8 = t[7:0];
    end
endmodule

// Pick the new 8-bit pin on the sample cycle, otherwise hold.
module sample_mux (
    input  wire       sample,
    input  wire [7:0] hold,
    input  wire [7:0] captured,
```

```
    output wire [7:0] d
);
    assign d = sample ? captured : hold;
endmodule

// Next state and next iteration. Combinational only.
module ctrl_next (
    input  wire [1:0] state,
    input  wire [2:0] i,
    input  wire       start,
    output reg  [1:0] state_next,
    output reg  [2:0] i_next,
    output wire       sample
);
    localparam IDLE   = 2'd0;
    localparam LOAD   = 2'd1;
    localparam RUN    = 2'd2;
    localparam SAMPLE = 2'd3;

    assign sample = (state == SAMPLE);

    always @* begin
        state_next = state;
        i_next     = i;
        case (state)
            IDLE: begin
                if (start) begin
                    state_next = LOAD;
                    i_next     = 3'd0;
                end
            end
            LOAD: state_next = RUN;
            RUN: begin
                if (i == 3'd7)
                    state_next = SAMPLE;
                else
                    i_next = i + 3'd1;
            end
            SAMPLE: state_next = IDLE;
```

```
            default: state_next = IDLE;
        endcase
    end
endmodule
```

B Testbench đầy đủ

`tb_cordic.v` — testbench của lõi, in `I05 RTL TB PASS`

```
‘timescale 1ns/1ps
// Same check as i05_cordic_core_tb.sv. Stored integers, pass band +/- 4.
// Port names follow this design: sin, cos, done. There is no busy pin.
module tb_cordic;
    reg clk = 0, rst_n = 0, start = 0;
    reg [7:0] angle;
    wire signed [7:0] sin, cos;
    wire done;

    cordic dut (
        .clk   (clk),
        .rst_n (rst_n),
        .start (start),
        .angle (angle),
        .sin   (sin),
        .cos   (cos),
        .done  (done)
    );

    always #5 clk = ~clk;

    task run_case;
        input [7:0] p;
        input integer sin_expected;
        input integer cos_expected;
        integer ds, dc;
        begin
            @(negedge clk);
            angle = p;
            start = 1’b1;

            @(negedge clk);
            start = 1’b0;

            wait (done);
```

```
            ds = sin - sin_expected;
            dc = cos - cos_expected;

            if ((ds > 4) || (ds < -4))
                $fatal(1, "sin mismatch p=%0d got=%0d exp=%0d", p, sin, sin_expected)
            if ((dc > 4) || (dc < -4))
                $fatal(1, "cos mismatch p=%0d got=%0d exp=%0d", p, cos, cos_expected)
        end
    endtask

    initial begin
        angle = 0;

        repeat (2) @(posedge clk);
        rst_n = 1;

        run_case(8'd0,    0,   127);
        run_case(8'd64,   127,   0);
        run_case(8'd128,  0,  -128);
        run_case(8'd192, -128,   0);
        run_case(8'd32,   91,   91);
        run_case(8'd224, -91,   91);

        $display("I05 RTL TB PASS");
        $finish;
    end
endmodule
```

`tb_tt_um_cordic.v` — testbench của vỏ chân, in `I05 PIN SHELL PASS`

```
`timescale 1ns/1ps
// Same six cases and the same +/- 4 band as i05_cordic_core_tb.sv,
// read back through the charter pins.
module tb_tt_um_cordic;
    reg clk = 0, rst_n = 0, ena = 1;
    reg [7:0] ui_in;
    reg [7:0] uio_in;
    wire [7:0] uo_out, uio_out, uio_oe;
```

```
tt_um_cordic dut (
    .ui_in   (ui_in),
    .uo_out  (uo_out),
    .uio_in  (uio_in),
    .uio_out (uio_out),
    .uio_oe  (uio_oe),
    .ena     (ena),
    .clk     (clk),
    .rst_n   (rst_n)
);

always #5 clk = ~clk;

task run_case;
    input [7:0] p;
    input integer sin_expected;
    input integer cos_expected;
    integer ds, dc;
    begin
        @(negedge clk);
        ui_in  = p;
        uio_in = 8'b0000_0001;

        @(negedge clk);
        uio_in = 8'b0;

        wait (uio_oe == 8'hff);

        ds = $signed(uo_out) - sin_expected;
        dc = $signed(uio_out) - cos_expected;

        if ((ds > 4) || (ds < -4))
            $fatal(1, "sin mismatch p=%0d got=%0d exp=%0d", p, $signed(uo_out), si
        if ((dc > 4) || (dc < -4))
            $fatal(1, "cos mismatch p=%0d got=%0d exp=%0d", p, $signed(uio_out), c

        @(posedge clk);
        @(negedge clk);
```

```
            if (uio_oe != 8'h00)
                $fatal(1, "uio stayed an output after done, p=%0d oe=%b", p, uio_oe);
        end
    endtask

    initial begin
        ui_in = 0;
        uio_in = 0;

        repeat (2) @(posedge clk);
        rst_n = 1;

        run_case(8'd0,    0,   127);
        run_case(8'd64,   127,   0);
        run_case(8'd128,  0,  -128);
        run_case(8'd192, -128,   0);
        run_case(8'd32,   91,   91);
        run_case(8'd224, -91,   91);

        $display("IO5 PIN SHELL PASS");
        $finish;
    end
endmodule
```

tb_error_dump.v — đổ sai số của mười mã

```
`timescale 1ns/1ps
// Print chip sin/cos for the error table. This file does not replace tb_cordic.v.
// The true sin/cos codes are computed in error_table.py. No pi in this bench.
module tb_error_dump;
    reg clk = 0, rst_n = 0, start = 0;
    reg [7:0] angle;
    wire signed [7:0] sin, cos;
    wire done;

    cordic dut (
        .clk   (clk),
        .rst_n (rst_n),
        .start (start),
```

```
    .angle (angle),
    .sin   (sin),
    .cos   (cos),
    .done  (done)
);

always #5 clk = ~clk;

task dump_case;
    input [7:0] p;
    begin
        @(negedge clk);
        angle = p;
        start = 1'b1;
        @(negedge clk);
        start = 1'b0;
        wait (done);
        $display("CHIP %0d %0d %0d", p, sin, cos);
    end
endtask

initial begin
    angle = 0;
    repeat (2) @(posedge clk);
    rst_n = 1;

    dump_case(8'd0);
    dump_case(8'd21);
    dump_case(8'd32);
    dump_case(8'd43);
    dump_case(8'd64);
    dump_case(8'd96);
    dump_case(8'd128);
    dump_case(8'd192);
    dump_case(8'd224);
    dump_case(8'd255);

    $finish;
end
```

```
endmodule
```

tb_gtkwave.v — một lần tính mã 32 cho dạng sóng

```
‘timescale 1ns/1ps
// One sample for GTKWave. Angle code 32 is 45 degrees.
// States: 0 Cho, 1 Nap, 2 Quay, 3 Lay mau.
// Open gtkwave on gtkwave/fsm_45.vcd and show:
//   clk start rst_n state i x y z sigma sin cos done
module tb_gtkwave;
    reg clk = 0, rst_n = 0, start = 0;
    reg [7:0] angle;
    wire signed [7:0] sin, cos;
    wire done;

    cordic dut (
        .clk   (clk),
        .rst_n (rst_n),
        .start (start),
        .angle (angle),
        .sin   (sin),
        .cos   (cos),
        .done  (done)
    );

    wire [1:0] state = dut.u_ctrl.u_state.q;
    wire [2:0] i     = dut.u_ctrl.u_i.q;
    wire signed [15:0] x = dut.u_proc.u_a.x;
    wire signed [15:0] y = dut.u_proc.u_a.y;
    wire signed [9:0]  z = dut.u_proc.u_b.z;
    wire sigma = dut.u_proc.u_b.sigma;

    always #5 clk = ~clk;

    function [8*8-1:0] sname;
        input [1:0] s;
        begin
            case (s)
                2’d0: sname = "Cho";
```

```
                2'd1: sname = "Nap";
                2'd2: sname = "Quay";
                2'd3: sname = "Lay mau";
                default: sname = "?";
            endcase
        end
    endfunction

    initial begin
        $dumpfile("gtkwave/fsm_45.vcd");
        $dumpvars(0, tb_gtkwave);
        angle = 8'd32;
        repeat (2) @(posedge clk);
        rst_n = 1;
        @(negedge clk);
        start = 1'b1;
        @(negedge clk);
        start = 1'b0;
        wait (done);
        @(posedge clk);
        @(posedge clk);
        $finish;
    end

    always @(posedge clk) begin
        #1;
        $display("t=%0t state=%s i=%0d sigma=%0b x=%0d y=%0d z=%0d sin=%0d cos=%0d do
                 $time, sname(state), i, sigma, x, y, z, sin, cos, done, start);
    end
endmodule
```

C Thông tin tái lập

Đường dẫn kho GitHub	`https://github.com/haiminhtruongisavailable/I05_Intro_to_chip_design_project`
Mã commit dùng cho báo cáo	`0520dd2`
Số hiệu / đường dẫn lần chạy CI	không có Actions trên kho này
Phiên bản LibreLane	OpenLane image `ff5509f65b17`, lệnh `./flow.tcl -design cordic`
Phiên bản PDK	`sky130A`
Ràng buộc chu kỳ xung nhịp	10 ns, trễ chân 2 ns
Ngày chạy	03/10/2026, bảng sai số 04/10/2026
Công cụ trí tuệ nhân tạo (AI) đã dùng và dùng vào việc gì	mô hình ngôn ngữ viết câu từ nhật ký lệnh của mạch này. Số liệu lấy từ lần chạy đã ghi.

Cam kết

Tôi xin cam đoan báo cáo này do tôi tự thực hiện. Toàn bộ số liệu trong báo cáo lấy từ lần chạy trên kho GitHub cá nhân nêu ở trang bìa và có thể đối chiếu được. Tôi hiểu rằng học phần này không thực hiện chế tạo chip, và các kết quả trong báo cáo là kết quả chạy công cụ mô phỏng và tổng hợp, không phải kết quả đo trên silicon.

Ngày ... tháng ... năm 20...

Người thực hiện

(Ký và ghi rõ họ tên)